

P1820r0: Recommendations for a compromise on handling errors and cancellations in executors

Date: 2019-10-07

Audience: SG1, LEWG

Authors: Gordon Brown

Emails: gordon@codeplay.com

Reply to: gordon@codeplay.com

Acknowledgements

Thanks to Chris Kohlhoff, Jamie Allsop, David Hollman, Eric Niebler, Kirk Shoop, Lewis Baker, Lee Howes, Jared Hoberock, Michał Dominiak and Christopher Di Bella for their helpful feedback.

Preface

This paper was originally written with the aim of resolving two conflicting positions on how to handle errors and cancellations in executors presented in [P1660](#) and [P1791](#) at the Cologne 2019 meeting. However, many of the recommended changes have now been applied in [P0443r11](#).

1. Motivation

[P1660](#) proposes a mechanism for handling errors and cancellations by which work items are represented by a `callback` which in addition to being an `invocable` also provides the `error` and `done` member functions for signalling errors and cancellations respectively. This approach has the benefit of allowing errors and cancellations to be propagated back via a separate channel from the value channel, even through different execution contexts. However it requires these channels to be provided every time a work item is enqueued.

[P1791](#) proposes an alternative mechanism for handling errors by which errors are handled by a callback associated with the executor via the `on_error_t` property, allowing the callback that handles errors to be specified only does not require work items to be defined as a `callback`. This approach is also useful for when you want errors to be handled by the execution context and maps closer to execution contexts such as OpenCL or CUDA which cannot always identify which work item triggered the error.

[P1660](#) provides a way to couple tasks with error and cancellation handling via the `callback` concept, however there exists domains where it's desirable to invoke functions which simply model `invocable` and handle errors and cancellations via some out-of-band mechanism such as calling `std::terminate`, propagating to some back-channel (provided either by the user or by the execution context) or even leaving the errors unhandled. The approach suggested in [P1660](#) does allow errors to be handled via alternative mechanisms however those mechanisms are currently entirely implementation defined, leaving error handling in the domains described above as unsupported in this model, which subsequently leaves these domains as unsupported.

Furthermore, there are some domains, such as GPU compute, where arbitrary callbacks to the host from a GPU driver would be detrimental to performance. However, there are algorithms expected to be defined with the `callback` concept as a basis operation such as those which define arbitrary error and cancellation handling channels which would require exactly this kind of arbitrary host callbacks. For execution contexts in such domains the only solution is to either ignore the semantic expectations of the algorithm or use a completely alternate set of algorithms, both of which are not acceptable solutions for a general executor solution.

In the case of a domain that would prefer to implement error and cancellation handling via an alternative mechanism to `callback`s, if a `callback` based approach was required by the basis operations this would introduce an unavoidable burden on implementation overhead to handling error in cases where it is necessary to type-erase the tasks, even if errors could be handled in an alternate mechanism.

Another consideration on code share-ability, is that since `sender`s (as described by [P1660](#)) are required to be specialized per execution context in order to be implemented efficiently, any algorithms which are defined with the `callback` concept as their basis operation cannot support executors which cannot or choose not to handle errors via `callback`s. This will ultimately mean codebases written to such algorithms may have to be re-written when being ported to run on different execution contexts, which is a concern for generic programming.

2. Summary

Throughout the discussion on this topic a number of recommendations have been identified:

Recommendation	Status in P0443r11
1. The <code>invocable</code> and <code>callback</code> concepts are independent and not part of a subsumption hierarchy.	Adopted as <code>invocable</code> and <code>receiver</code> concepts.
2. An algorithm or control structure can be constrained on <code>invocable</code> , and not be expected to support types which model <code>callback</code> .	Adopted via <code>executor</code> -based algorithms.
3. An algorithm or control structure can be constrained on <code>callback</code> and must honour the <code>callback</code> type’ s error an done channels.	Adopted via <code>sender</code> -based algorithms.
4. An executor can opt to support only an execution function which accepts <code>invocable</code> .	Adopted as <code>executor</code> concept.
5. An executor can opt to support only an execution function which accepts <code>callback</code> .	Adopted as <code>sender</code> concept.
6. The error and cancellation handling behavior of an execution function which accepts <code>invocable</code> should be well-defined via properties.	Not adopted.

3. Suggested change: executor properties for error handling and cancellation

To allow an executor to specify alternative error and cancellation mechanisms this paper proposes that P0443 adopt two new groups of behavioral properties (as defined by P0443), `execution::invocable_error_channel_t` and `execution::invocable_cancellation_channel_t` , which can be used to specify to an executor how to handle errors when a work item is specified with an `invocable` rather than a `callback` .

This paper currently proposes a limited number of properties for `execution::invocable_error_channel_t` and `execution::invocable_cancellation_channel_t` , however it is expected that these property groups be extended to incorporate other error and cancellation mechanism in the future as requirements for them arise.

execution::invocable_error_channel_t

The `execution::invocable_error_channel_t` properties is a group of mutually exclusive behavioral properties that specify the guarantee which an Executor makes as to how it responds to an error of a work item being executed.

For some Executor type `E` , when a work item being executed by an execution agent created by `E` results in an error `err` , `E` must:

- If the work-item was submitted as a `callback` `cb` , call `cb.error(err)` .
- Else, if the work-item was submitted as an `invocable` `i` , behave as-if calling a default constructed `callback` with `i` as its value channel and a default error channel who’ s behavior is specified by `execution::invocable_error_channel_t` .

Property type	Property object	Description
<code>execution::invocable_error_channel_t::terminate_t</code>	<code>execution::invocable_error_channel.terminate</code>	The error channel of the default <code>callback</code> will invoke <code>std::terminate()</code> .
<code>execution::invocable_error_channel_t::back_channel_t</code>	<code>execution::invocable_error_channel.back_channel(i)</code>	The error channel of the default <code>callback</code> behaves as-if calling <code>back_channel(i)</code> where <code>i</code> is an <code>invocable</code> that invokes <code>error</code> . <code>execution::invocable_error_channel.back_channel_t</code> takes a parameter <code>bk</code> , where <code>bk</code> is an <code>invocable<Error></code> .

execution::invocable_cancellation_channel_t

The `execution::invocable_cancellation_channel_t` properties is a group of mutually exclusive behavioral properties that specify the guarantee which an Executor makes as to how it responds to a cancellation of a work item being executed.

For some Executor type `E` :

Property type	Property object	
<code>execution::invocable_cancellation_channel_t::unhandled_t</code>	<code>execution::invocable_cancellation_channel.unhandled</code>	When a work does not prop
<code>execution::invocable_cancellation_channel_t::back_channel_t</code>	<code>execution::invocable_cancellation_channel.back_channel(i)</code>	When a work propagates th is an invocab <code>execution::i</code> takes a param

Status in P0443

This recommendation was not adopted into [P0443r11](#).

4. Suggested change: separate executor concepts for `execute(invocable)` and `execute(callback)`

In order to allow executor authors freedom to choose the level of support they want for their executors (i.e. support for types which model `callback` or types which model `invocable`), whilst also allowing algorithm authors to specify constraints on the kind of executors required this paper proposes that P0443 introduces a subsumption hierarchy of executor types taking the form of `invocable_executor` and `callback_executor` as a refinement of `invocable_executor` .

[Note: The names of these concepts are placeholder names and are expected to be bikeshed.]

`invocable_executor` concept

The `invocable_executor` concept requires a type to have a `.execute` member function which takes an `invocable` which handles errors and continuations according to the value of the `execution::invocable_error_channel_t` and `execution::invocable_cancellation_channel_t` properties for the executor respectively.

```
template<class T, class F = void(*)()>
concept invocable_executor = invocable<F> &&
    requires (T&& t, F&& f) {
        std::forward<T>(t).execute(std::forward<F>(f));
    };
```

`callback_executor` concept

The `callback_executor` concept requires a type to model the `invocable_executor` concept and to have a further `.execute` member function which takes a `callback` which handles errors and cancellations via the `error` and `done` channels of the `callback` as defined in [P1660](#).

```
template<class T, class C = __callback>
concept callback_executor = invocable_executor<T> ||
    (callback<C> && requires (T&& t, C&& c) {
        std::forward<T>(t).execute(std::forward<C>(c));
    });
```

[Note: `__callback` is a type used for exposition purposes only.]

Status in P0443

This recommendation was adopted into [P0443r11](#) with the concepts `executor` which is required to provide `execute(invocable)` and `sender` which is required to provide `submit(callback)` as separate types rather than a subsumption hierarchy, and minus the requirement to support on
generated by [haroopad](#)

`executor` to perform error handling according to the `execution::invocable_error_channel_t` and `execution::invocable_cancellation_channel_t` properties.

5. Suggested change: customization point `execution::execute`

In order to allow adaptations between a `invocable_executor` and a `callback_executor` this paper recommends that P0443 provide the `execution::execute` customization point object.

For a given executor `e` of type `E` and a given function `f` of type `F`, `execution::execute(e, f)` should behave as follows:

	<code>E</code> models <code>invocable_executor</code>	<code>E</code> models <code>callback_executor</code>
<code>F</code> models <code>invocable</code>	Invokes <code>execute(invocable &&)</code> . Errors and cancellations are handled according to the value of the <code>execution::invocable_error_channel_t</code> and <code>execution::invocable_cancellation_channel_t</code> properties for <code>E</code> respectively.	Invokes <code>execute(invocable &&)</code> . Errors and cancellations are handled according to the value of the <code>execution::invocable_error_channel_t</code> and <code>execution::invocable_cancellation_channel_t</code> properties for <code>E</code> respectively.
<code>F</code> models <code>callback</code>	Compile-time error.	Invokes <code>execute(callback &&)</code> . Errors and cancellations are handled by <code>F::error</code> and <code>F::done</code> respectively.

The one caveat to the semantic relationship between executor type and function type is that as `invocable` and `callback` are a subsumption hierarchy, calling `execution::execute` with an executor type that models `invocable_executor` and a function type which models `callback` will result in the `error` and `done` channels of the `callback` not being used. However if this is a problem for algorithm authors for any particular algorithm this can be avoided by simply constraining the algorithm on `callback_executor` therefore preventing the user from using a type which only models `invocable_executor`.

[Note: The above definitions of `invocable_executor` and `callback_executor` are for exposition purposes, and the `callback_executor` concept can also be implemented as a single overload which handles both cases.]

Status in P0443

This recommendation was adopted into P0443r11 with the customization points `tbd::execute(executor, invocable)` which is required to provide the semantics of `execute(invocable)` and `tbd::submit(sender, callback)` which is required to provide the semantics of `submit(callback)`, and minus the requirement to support on `execute(executor, invocable)` to perform error handling according to the `execution::invocable_error_channel_t` and `execution::invocable_cancellation_channel_t` properties.

6. Suggested change: Separately constrained algorithms and control structures

By introducing the subsumption hierarchy of executor types; `invocable_executor` and `callback_executor` algorithm authors can allow an algorithm to be generic to the executor type or enforce strict requirements on whether only types which model `callback` can be used.

Algorithm constraints	Expected behavior
<pre>template <class S, class F> requires invocable_executor<typename S::executor_type> auto transform(S sender, F func);</pre>	Generic algorithm which can take an executor that models either <code>invocable_executor</code> or <code>callback_executor</code> behaving appropriately to what is passed, returning a sender which may use <code>callback</code> or may use <code>execution::invocable_error_channel_t</code> and <code>execution::invocable_cancellation_channel_t</code> .
<pre>template <class S, class F, class E> requires callback_executor<typename S::executor_type> auto transform_error(S sender, F func, E errorFunc);</pre>	<code>callback</code> specific algorithm which can only take an executor that models either <code>callback_executor</code> and therefore must return a sender that uses <code>callback</code> .

Status in P0443

It is the intended direction of P0443 is to adopt algorithms such as those defined above, however [P0443r11](#) does not yet specify these.